فهرس الفصول (هيكل Word باللغة العربية):

☑ الفصل الأول: مقدمة عن المشروع

- نبذة عامة عن المشروع.
- أهمية المشروع ولماذا تم اختياره.
- أهداف المشروع.
- نطاق المشروع.

☑ الفصل الثاني: الأدوات والتقنيات المستخدمة

- لغة البرمجة: PHP
- إطار العمل: Laravel
- مكتبة التصميم: Tailwind CSS
- قاعدة البيانات: MySQL
- أدوات إضافية: Composer, VS Code, Git...

☑ الفصل الثالث: تحليل النظام

- وصف المستخدمين (مثل: مستخدم عادي، مشرف).
- متطلبات النظام (الوظيفية وغير الوظيفية).
- حالات الاستخدام (Use Case Diagram).
- مخطط تدفق البيانات (DFD) - إن وُجد.

☑ الفصل الرابع: تصميم قاعدة البيانات

- شرح الجداول والعلاقات.
- المخطط التصوري (ERD).
- نموذج أولي للجداول: أسماء الأعمدة، أنواع البيانات، المفاتيح الأساسية/الخارجية.

☑ الفصل الخامس: التصميم والبرمجة

- هيكل المشروع (مجلدات Laravel).

- شرح ملفات Routes.
- شرح Controllers.
- نماذج Blade Views.
- مقاطع كود مع شرح وظيفتها.

☑ الفصل السادس: واجهات المستخدم (Screenshots)

- صور لكل صفحة:
  - تسجيل الدخول.
  - صفحة الكابتشا.
  - لوحة التحكم.
  - أي صفحات إضافية.
- وصف لكل واجهة ووظيفتها.

☑ الفصل السابع: طريقة التشغيل والاستخدام

- خطوات تثبيت وتشغيل المشروع:
  - clone المشروع.
  - `composer install`.
  - إعداد قاعدة البيانات وملف `.env`.
  - `php artisan migrate`.
  - تشغيل الخادم: `php artisan serve`.
- كيفية تجربة النظام من البداية.

☑ الفصل الثامن: التحديات والحلول

- المشاكل التي واجهتها أثناء التطوير.
- كيف تم تجاوزها.
- دروس مستفادة.

☑ الفصل التاسع: الخاتمة والتوصيات

- تلخيص المشروع.
- ما الذي يمكن تطويره مستقبلاً؟
- توصيات للمطورين أو للجامعة.

شروع نحل العسل باستخدام Laravel وTailwind CSS

الفصل الأول: مقدمة عن المشروع

مشروع "نحل العسل" هو نظام ويب تم تطويره باستخدام Laravel وTailwind CSS، يهدف إلى توفير واجهة تفاعلية لإدارة المستخدمين والتحقق من الهوية عبر كود CAPTCHA. يساعد هذا النظام في تعزيز الأمان عند الدخول، ويُستخدم كجزء من متطلبات مشروع تخرج جامعي.

الفصل الثاني: الأدوات والتقنيات المستخدمة

- **اللغة الأساسية:** PHP 8.x
- **الإطار المستخدم:** Laravel 10
- **تصميم الواجهات:** Tailwind CSS
- **قاعدة البيانات:** MySQL
- **أدوات إضافية:**
  - Composer
  - Git / GitHub
  - Visual Studio Code
  - XAMPP أو Laravel Sail

الفصل الثالث: تصميم قاعدة البيانات

يعتمد النظام على قاعدة بيانات بسيطة لإدارة المستخدمين. ومن الجداول الأساسية:

جدول المستخدمين:

```
php
```

```
CopyEdit
Schema::create('users', function (Blueprint
$table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->string('password');
    $table->timestamps();
});
```

يتم استخدام التحقق من الكابتشا عبر جلسة مؤقتة للمستخدم قبل منحه الوصول النهائي للنظام.

📃 الفصل الرابع: مقتطفات من الكود

📌 routes/web.php

```
php
CopyEdit
Route::get('/login', [LoginController::class,
'showLoginForm'])->name('login');
Route::post('/login', [LoginController::class,
'login']);
Route::get('/captcha-image',
[CaptchaController::class, 'generateCaptcha']);
Route::post('/verify-captcha',
[CaptchaController::class, 'verify']);
```

📌 LoginController.php

```
php
CopyEdit
public function login(Request $request) {
    $credentials = $request->only('email',
'password');
```

```
    if (Auth::attempt($credentials)) {
        session(['pending_user_id' =>
Auth::id()]);
        return redirect('/verify-captcha');
    }
    return back()->withErrors(['email' =>
'Invalid credentials']);
}
```

📌 CaptchaController.php

```
php
CopyEdit
public function generateCaptcha()
{
    $captcha = new CaptchaBuilder;
    $captcha->build();
    session(['captcha' => $captcha-
>getPhrase()]);
    return response($captcha->inline())-
>header('Content-Type', 'image/jpeg');
}

public function verify(Request $request)
{
    if ($request->captcha ===
session('captcha')) {

Auth::loginUsingId(session('pending_user_id'));
        return redirect()->route('home.index');
    }

    return back()->withErrors(['captcha' =>
'رمز التحقق غير صحيح']);
}
```

👨‍💻 الفصل الخامس: طريقة التشغيل

1. تحميل المشروع أو نسخه من GitHub.
2. تشغيل الأمر: `composer install`
3. إعداد ملف `.env` وربط قاعدة البيانات.
4. تنفيذ أوامر:
   - `php artisan migrate`
   - `php artisan serve`
5. الدخول إلى المسار: `http://localhost:8000/login`

🎨 الفصل السادس: الواجهات

تحتوي واجهات المشروع على:

- شاشة تسجيل دخول بتصميم أنيق.
- شاشة تحقق من كود CAPTCHA.
- زر 🔄 لتحديث الصورة يدويًا.
- استخدام ألوان خضراء هادئة لتمثيل الطبيعة و"العسل".

📌 *(يُفضل إدراج صور للواجهات هنا: login page – captcha page... إلخ)*

⚠️ الفصل السابع: التحديات

- التعامل مع توليد الكابتشا وربطها بالجهاز عبر الجلسة.
- تحسين تجربة المستخدم بالتصميم المتجاوب عبر Tailwind.
- التأكد من توافق النظام مع المتصفحات المختلفة.

☑ الفصل الثامن: الخاتمة

تم فيه Laravel. مشروع "نحل العسل" يُعد نموذجًا عمليًا لنظام تسجيل دخول آمن باستخدام توظيف العديد من مفاهيم تطوير الويب الحديثة. ويمكن تطويره لاحقًا ليشمل:

- لوحة تحكم للمستخدمين.
- إدارة منتجات العسل.
- واجهات متعددة اللغات.
- استخدام تقنيات أكثر تقدمًا مثل Vue.js.

الفصل التاسع: تحليل قاعدة البيانات ومخطط الكيانات (ERD)

📌 تحليل قاعدة البيانات

قاعدة البيانات في مشروع "نحل العسل" بسيطة ولكنها قابلة للتوسعة. إليك تحليلًا لأهم الجداول:

🔶 جدول `users` (المستخدمين)

الحقل	النوع	الوصف
id	BIGINT	رقم تعريفي تلقائي
name	VARCHAR	اسم المستخدم
email	VARCHAR	البريد الإلكتروني (فريد)
password	VARCHAR	كلمة المرور (مشفرّة)
created_at	TIMESTAMP	تاريخ الإنشاء
updated_at	TIMESTAMP	تاريخ التعديل

🔶 جدول الجلسات المؤقتة (باستخدام session)

- مؤقتًا (Session) لا يتم تخزين الكابتشا في جدول خاص، بل في الجلسة باستخدام:

```
php
CopyEdit
session(['captcha' => $captcha-
>getPhrase()]);
session(['pending_user_id' => $user-
>id]);
```

📈 التمثيل النصي - مخطط الكيانات (ERD)

```
pgsql
CopyEdit
```

